HYPERTENSION RISK PREDICTION SYSTEM

PROJECT REPORT

NAME: ADITI SAHU

REGISTRATION NO.: 25BCE10844

BRANCH: COMPUTER SCIENCE AND ENGINEERING

YEAR: FIRST YEAR

COURSE: FUNDAMENTALS OF AI & ML

INTRODUCTION

HYPERTENSION RISK PREDICTION SYSTEM is a machine learning model, developed to predict risk of hypertension based on user health and lifestyle. It takes input of parameters such as age, BMI, stress level, and exercise habits and predicts whether a person is at High Risk or Low Risk of having hypertension. It should be noted that this is an academic project to demonstrate the application of machine learning techniques and not a medical tool.

Hypertension (high blood pressure) is a major health concern that can lead to serious diseases. Many people have this condition but don't know it because it usually has no symptoms.Without treatment, it can lead to a heart attack, stroke, dementia, kidney disease and other issues. **Early detection can help in prevention and better treatment.**

Hypertension Risk Prediction System can predict the risk of hypertension and classify it as high or low based on the user's input and also report the probability percentage of risk, which can alarm an individual if they need any immediate medical assistance.

As a part of fulfillment of my course of Fundamentals in AI-ML, I was required to complete a **Bring Your Own Project.** I had to identify a real-world problem and solve it using the concepts, tools, and techniques covered in the course. So, I came up with the idea of building a Hypertension Risk Prediction System. The reason behind my chosen problem is my own personal experience where hypertension really acted as a "silent killer" and I realised the importance of early risk prediction of risk in such cases.

This hypertension risk prediction system is developed purely as an academic project to demonstrate the application of machine learning techniques. The data used for training and testing is synthetic (sourced from Kaggle) and does not represent real medical records. The outputs generated by this system are for educational purposes only and should not be interpreted as medical advice, diagnosis, or a substitute for professional healthcare.

PROBLEM STATEMENT

As a part of **Bring your own project,** the problem I aim to solve is development of a machine learning-based classification model that can predict the risk of hypertension (binary classification: Yes/No) and report probability percentage using features such as age, salt intake, stress level, sleep duration, BMI, family history, exercise level, smoking status, blood pressure history, and medication.

The final output should is:

- A trained model
- A command-line interface (predict.py) for making new predictions
- Proper documentation and reproducibility

OBJECTIVES

- To search for suitable dataset required to train the model
- To preprocess the dataset containing mixed numerical and categorical features.
- To compare multiple classification algorithms (Logistic Regression, Random Forest, KNN, XGBoost).
- To select the best performing model (XGBoost in this case).
- To build a complete end-to-end ML pipeline.
- To create a user-friendly command-line interface.

DATASET DESCRIPTION

The dataset is collected from Kaggle. It contains **1,985 records** and 11 meaningful features.
Source-https://www.kaggle.com/datasets/miadul/hypertension-risk-prediction-dataset

Features (10 input features):

- Age (Numerical)
- Salt_Intake (Numerical)
- Stress_Score (Numerical)
- Sleep_Duration (Numerical)
- BMI (Numerical)
- BP_History (Categorical: Hypertension, Normal, Prehypertension)
- Medication (Categorical)
- Family_History (Yes/No)
- Exercise_Level (Low/Moderate/High)
- Smoking_Status (Smoker/Non-Smoker)

Target Variable: Has_Hypertension (Yes/No encoded as 1/0)

METHODOLOGY

The project was developed in the given steps following the standard Machine-Learning pipeline:

1. **DATA COLLECTION**
2. **DATA PREPROCESSING:**
    - Handled missing values - Missing values in column Medication filled with 'None'
    - Converted all categorical features into numerical form usingLabel Encoding
    - Train-Test Split - Dataset was split into training and testing sets (80:20) with stratification to maintain class balance.
3. **FEATURE SCALING:**
    - Feature Scaling was done using StandardScaler (applied to all features) as many ML models perform better with standardized data
4. **MODEL TRAINING:**
    - Multiple models were trained using XGBoost, Random Forest, Logistic Regression and K Nearest Neighbors (KNN) algorithms
5. **MODEL EVALUATION**
    - Accuracy, Precision, Recall, F1-Score, and ROC-AUC were evaluated for each model
    - Model evaluation was done on the basis of Accuracy, Precision, Recall, F1-Score, and ROC-AUC
6. **MODEL COMPARISON & FINAL MODEL SELECTION:**
    - After evaluation, all the trained models were compared and XGBoost was selected as the final model.
    - XGBoost was selected as it has highest F1-Score (0.9783), highest ROC-AUC (0.9984), highest Accuracy (0.9773) and very balanced precision and recall.

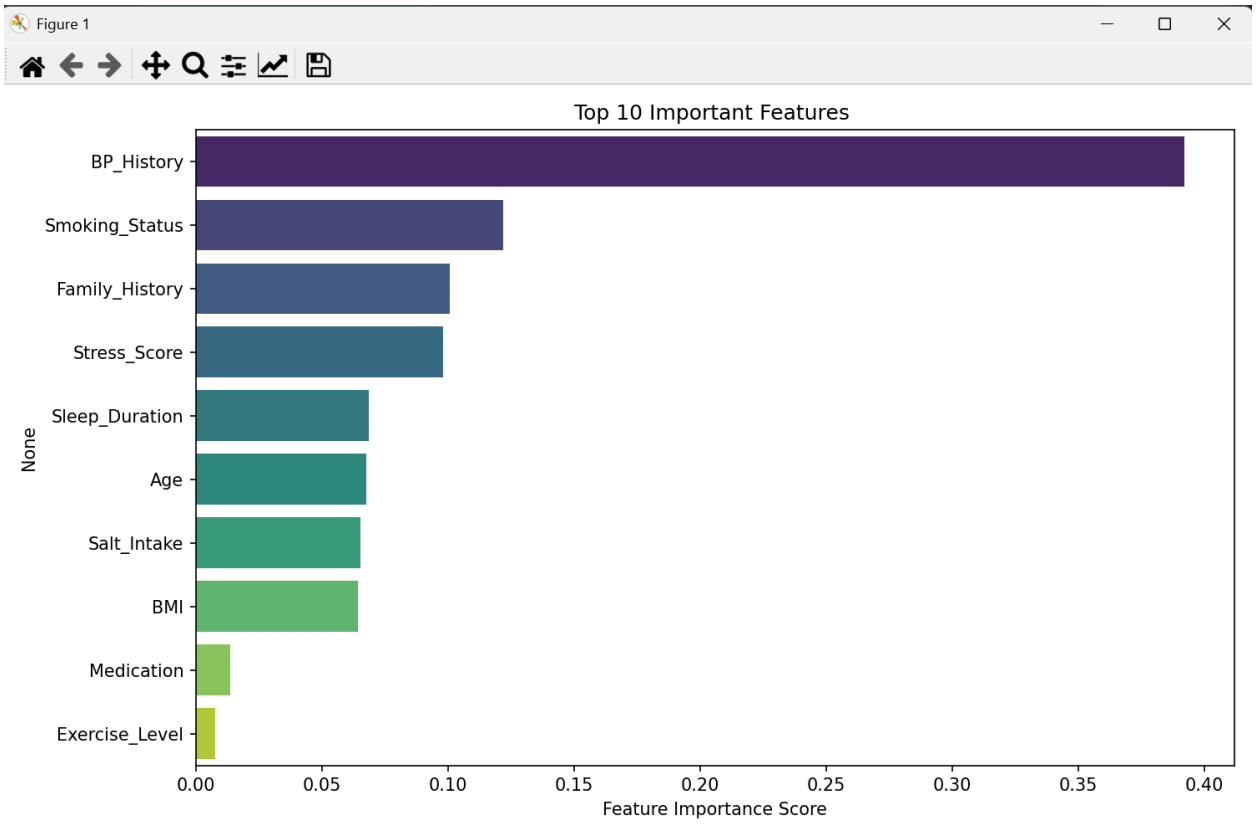
      Note- The XGBoost Model was trained twice. XGBoost achieved the highest F1-score of 0.9783 during model comparison. However, the final model training resulted in slightly different performance (F1-Score (0.9663), ROC-AUC (0.9968), Accuracy (0.9647)). The reason behind this variation is randomness in XGBoost, or small differences in how it's initialized.But it still remains the best model as it shows consistent performance depending on training run.

7. **MODEL SAVING:**
    - The final model was saved using **Joblib**
8. **COMMAND LINE INTERFACE:**
    - predict.py was written for inference via command line

MODEL PERFORMANCE

- XGBoost Classifier - The final model
- Hyperparameters: n_estimators=200, learning_rate=0.1, max_depth=6
- XGBoost handles mixed data types and non-linear relationships efficiently and hence outperforms other models.
- **Performance Metrics:**
  - Accuracy: 0.9647
  - F1-Score: 0.9663
  - ROC-AUC: 0.9968
- Feature Importance: BP_History, Smoking_Status, Family_History, Stress_Score and Sleep_Duration are the most influential factors. This Feature Importance explains the "why" behind Hypertension Risk Prediction System's decisions.

Feature Importance of the model



RESULTS

train.py (the attached image includes code for data preprocessing, model training, model evaluation, model comparison, model saving and feature importance)

train.py - C:\ai-ml\src\train.py (3.14.1)

File Edit Format Run Options Window Help

```python
# IMPORTS
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import os
import joblib
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from xgboost import XGBClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score, confusion_matrix, classification_report

#  MAIN FUNCTION
def main():
    print(" Loading dataset")
    df = pd.read_csv("data/hypertension_dataset.csv")

    #  DATA PREPROCESSING
    df['Medication'] = df['Medication'].fillna('None')
    df['Has_Hypertension'] = df['Has_Hypertension'].map({'Yes': 1, 'No': 0})

    categorical_cols = ['BP_History', 'Medication', 'Family_History',
                        'Exercise_Level', 'Smoking_Status']

    le_dict = {}
    for col in categorical_cols:
        le = LabelEncoder()
        df[col] = le.fit_transform(df[col])
        le_dict[col] = le

    X = df.drop('Has_Hypertension',axis=1)
    y = df['Has_Hypertension']

    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)

    #  MODEL TRAINING
    print("\n Training models...\n")
    models = {
        "Logistic Regression": LogisticRegression(max_iter=1000, random_state=42),
        "Random Forest": RandomForestClassifier(n_estimators=200, random_state=42),
```

Ln: 1 Col: 0

train.py - C:\ai-ml\src\train.py (3.14.1)

File Edit Format Run Options Window Help

```python
    X_test_scaled = scaler.transform(X_test)

    #  MODEL TRAINING
    print("\n Training models...\n")
    models = {
        "Logistic Regression": LogisticRegression(max_iter=1000, random_state=42),
        "Random Forest": RandomForestClassifier(n_estimators=200, random_state=42),
        "KNN": KNeighborsClassifier(n_neighbors=5),
        "XGBoost": XGBClassifier(eval_metric='logloss', random_state=42)
    }

    results = []
    for name, model in models.items():
        if name in ["Logistic Regression", "KNN"]:
            model.fit(X_train_scaled, y_train)
            y_pred = model.predict(X_test_scaled)
            y_prob = model.predict_proba(X_test_scaled)[:, 1]
        else:
            model.fit(X_train, y_train)
            y_pred = model.predict(X_test)
            y_prob = model.predict_proba(X_test)[:, 1]

        acc = accuracy_score(y_test, y_pred)
        prec = precision_score(y_test, y_pred)
        rec = recall_score(y_test, y_pred)
        f1 = f1_score(y_test, y_pred)
        auc = roc_auc_score(y_test, y_prob)

        results.append({
            "Model": name,
            "Accuracy": round(acc, 4),
            "Precision": round(prec, 4),
            "Recall": round(rec, 4),
            "F1": round(f1, 4),
            "ROC-AUC": round(auc, 4)
        })
        print(f"{name} done")

    results_df = pd.DataFrame(results)
    print("\nModel Comparison (sorted by F1):")
    print(results_df.sort_values(by="F1",ascending=False))

    # FINAL MODEL (XGBoost) =
    print("\nTraining FINAL model (XGBoost)")
    final_model = XGBClassifier(
        eval_metric='logloss',
        random_state=42,
```

Ln: 1 Col: 0

train.py - C:\ai-ml\src\train.py (3.14.1)

File Edit Format Run Options Window Help

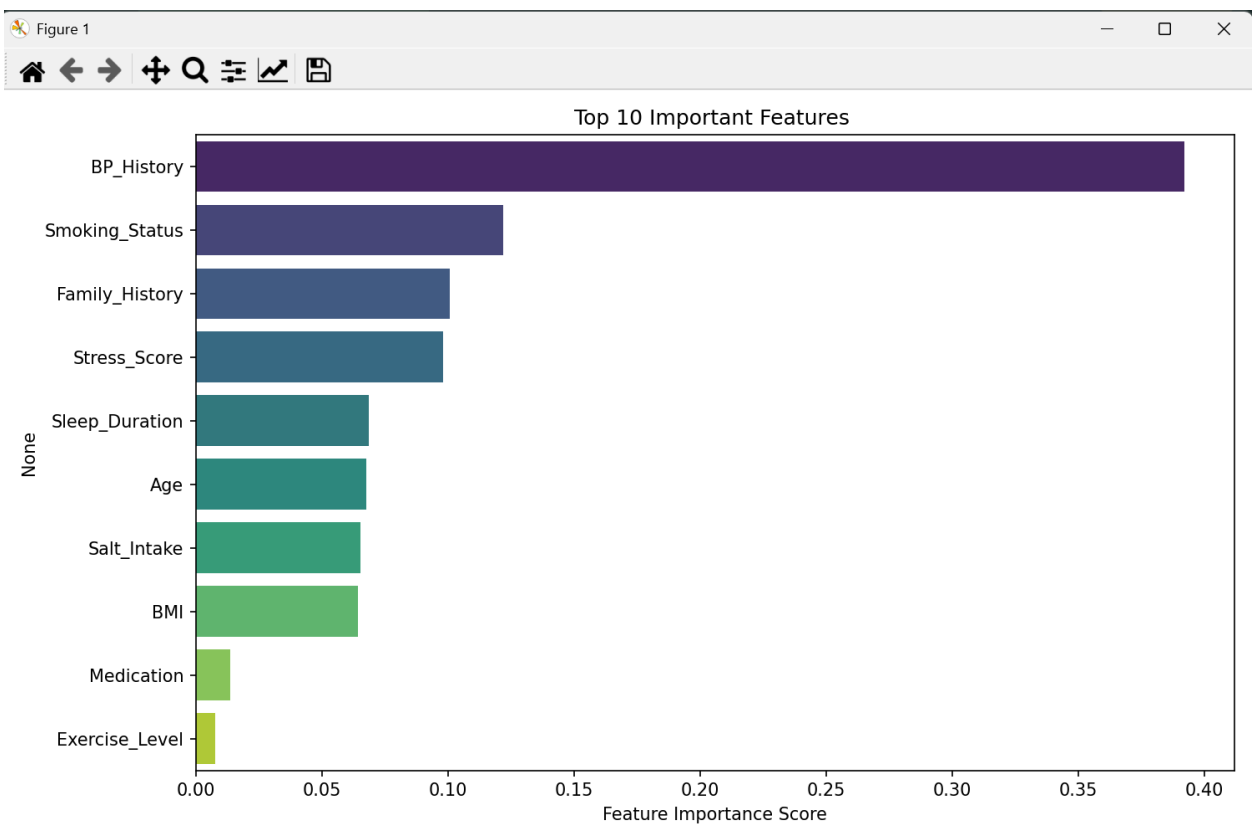
```python
        random_state=42,
        n_estimators=200,
        learning_rate=0.1,
        max_depth=6)
    final_model.fit(X_train, y_train)

    y_pred = final_model.predict(X_test)
    y_prob = final_model.predict_proba(X_test)[:, 1]

    print("\n=== FINAL MODEL PERFORMANCE ===")
    print("Accuracy :", round(accuracy_score(y_test, y_pred), 4))
    print("F1 Score :", round(f1_score(y_test, y_pred), 4))
    print("ROC-AUC :", round(roc_auc_score(y_test, y_prob), 4))
    print("\nClassification Report:")
    print(classification_report(y_test, y_pred))
    print("\nConfusion Matrix:")
    print(confusion_matrix(y_test, y_pred))

    # SAVE MODEL
    os.makedirs("model", exist_ok=True)
    joblib.dump(final_model, "model/hypertension_model.pkl")
    joblib.dump(scaler, "model/scaler.pkl")
    joblib.dump(le_dict, "model/encoders.pkl")
    joblib.dump(X.columns.tolist(), "model/columns.pkl")
    print("\n Model and Scaler saved successfully in 'model/' folder!")

    # = FEATURE IMPORTANCE
    importances = pd.Series(final_model.feature_importances_, index=X.columns)
    print("\n Top 10 Important Features:")
    print(importances.sort_values(ascending=False).head(10))

    plt.figure(figsize=(10, 6))
    sns.barplot(
        x=importances.sort_values(ascending=False).head(10),
        y=importances.sort_values(ascending=False).head(10).index,
        palette="viridis"
    )
    plt.title("Top 10 Important Features ")
    plt.xlabel("Feature Importance Score")
    plt.tight_layout()
    plt.show()


#  RUN
if __name__ == "__main__":
    main()
```

Result of train.py:

```
(base) C:\ai-ml>python src\train.py
 Loading dataset

 Training models...

Logistic Regression done
Random Forest done
KNN done
XGBoost done

Model Comparison (sorted by F1):
                 Model  Accuracy  Precision  Recall      F1  ROC-AUC
3              XGBoost    0.9773     0.9713  0.9854  0.9783   0.9984
1        Random Forest    0.9572     0.9522  0.9660  0.9590   0.9956
2                  KNN    0.8413     0.8950  0.7864  0.8372   0.9144
0  Logistic Regression    0.8212     0.8392  0.8107  0.8247   0.9157

Training FINAL model (XGBoost)

=== FINAL MODEL PERFORMANCE ===
Accuracy : 0.9647
F1 Score : 0.9663
ROC-AUC : 0.9968

Classification Report:
              precision    recall  f1-score   support

           0       0.97      0.95      0.96       191
           1       0.96      0.98      0.97       206

    accuracy                           0.96       397
   macro avg       0.97      0.96      0.96       397
weighted avg       0.96      0.96      0.96       397


Confusion Matrix:
```

```
Classification Report:
              precision    recall  f1-score   support

           0       0.97      0.95      0.96       191
           1       0.96      0.98      0.97       206

    accuracy                           0.96       397
   macro avg       0.97      0.96      0.96       397
weighted avg       0.96      0.96      0.96       397


Confusion Matrix:
[[182   9]
 [  5 201]]

 Model and Scaler saved successfully in 'model/' folder!

 Top 10 Important Features:
BP_History        0.392109
Smoking_Status    0.122020
Family_History    0.100589
Stress_Score      0.098136
Sleep_Duration    0.068741
Age               0.067701
Salt_Intake       0.065264
BMI               0.064310
Medication        0.013508
Exercise_Level    0.007623
dtype: float32
C:\ai-ml\src\train.py:119: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `leg
end=False` for the same effect.

  sns.barplot(

(base) C:\ai-ml>
```

Feature Importance



predict.py (command line interface)

predict.py - C:\ai-ml\src\predict.py (3.14.1)

File Edit Format Run Options Window Help

```python
import joblib
import pandas as pd

# Load saved files
model = joblib.load("model/hypertension_model.pkl")
encoders = joblib.load("model/encoders.pkl")
columns = joblib.load("model/columns.pkl")

print("====== Hypertension Risk Prediction System ======\n")

input_data = {}

# Take input for each column
for col in columns:
    if col in encoders:  # categorical column
        le = encoders[col]
        print(f"{col} (Options: {list(le.classes_)})")
        value = input(f"Enter {col}: ")
        try:
            value = le.transform([value])[0]
        except:
            print(f"Invalid value for {col}")
            exit()
    else:  # numerical column
        value = float(input(f"Enter {col}: "))

    input_data[col] = value

# Create dataframe
input_df = pd.DataFrame([input_data])

# Prediction
prediction = model.predict(input_df)[0]
prob = model.predict_proba(input_df)[0][1]

# Output
print("\n===== RESULT =====")
if prediction == 1:
    print("Hypertension Risk: HIGH")
else:
    print("Hypertension Risk: LOW")

print(f"Probability: {round(prob*100, 2)}%\nThis system is an academic machine learning project using synthetic Kaggle data — it is for educational purposes
```

predict.py - C:\ai-ml\src\predict.py (3.14.1)

File Edit Format Run Options Window Help

```python
ertension_model.pkl")
encoders.pkl")
olumns.pkl")

k Prediction System ======\n")

egorical column

: {list(le.classes_)})")
{col}: ")

rm([value])[0]

lue for {col}")

Enter {col}: "))

_data])

ut_df)[0]
ut_df)[0][1]

HIGH")

LOW")

ob*100, 2)}%\nThis system is an academic machine learning project using synthetic Kaggle data — it is for educational purposes only and not a medical tool.")
```

Ln: 43 Col: 187

```
(base) C:\ai-ml>python src\predict.py
====== Hypertension Risk Prediction System ======

Enter Age: 34
Enter Salt_Intake: 8
Enter Stress_Score: 9.5
BP_History (Options: ['Hypertension', 'Normal', 'Prehypertension'])
Enter BP_History: Hypertension
Enter Sleep_Duration: 5
Enter BMI: 25
Medication (Options: ['ACE Inhibitor', 'Beta Blocker', 'Diuretic', 'None', 'Other'])
Enter Medication: None
Family_History (Options: ['No', 'Yes'])
Enter Family_History: Yes
Exercise_Level (Options: ['High', 'Low', 'Moderate'])
Enter Exercise_Level: Low
Smoking_Status (Options: ['Non-Smoker', 'Smoker'])
Enter Smoking_Status: Smoker

===== RESULT =====
Hypertension Risk: HIGH
Probability: 99.97000122070312%
This system is an academic machine learning project using synthetic Kaggle data — it is for educational purposes only and not a medic
al tool.
```

```
Anaconda Prompt

(base) C:\Users\aditi>cd C:\ai-ml

(base) C:\ai-ml>python src\predict.py
====== Hypertension Risk Prediction System ======

Enter Age: 19
Enter Salt_Intake: 7
Enter Stress_Score: 8
BP_History (Options: ['Hypertension', 'Normal', 'Prehypertension'])
Enter BP_History: Prehypertension
Enter Sleep_Duration: 6
Enter BMI: 20.9
Medication (Options: ['ACE Inhibitor', 'Beta Blocker', 'Diuretic', 'None', 'Other'])
Enter Medication: None
Family_History (Options: ['No', 'Yes'])
Enter Family_History: Yes
Exercise_Level (Options: ['High', 'Low', 'Moderate'])
Enter Exercise_Level: Moderate
Smoking_Status (Options: ['Non-Smoker', 'Smoker'])
Enter Smoking_Status: Non-Smoker

===== RESULT =====
Hypertension Risk: LOW
Probability: 1.659999966621399%
This system is an academic machine learning project using synthetic Kaggle data — it is for educational purposes only and not a medic
al tool.

(base) C:\ai-ml>
```

"DISCLAIMER: This is a Machine Learning project for educational purposes only. The predictions generated are based on synthetic dataset from Kaggle and should NOT be used as medical advice, diagnosis, or a substitute for professional healthcare consultation."

CHALLENGES FACED & SOLUTIONS

During the development of Hypertension Risk Prediction System I encountered many challenges which pushed me to explore the technicalities of Machine Learning. I was stuck in many parts of the project, whether it is inconsistency in the model, scalar feature mismatch, etc. While handling these issues, I learned many important aspects of ML development which enhanced my overall skills in the related field.

- **Challenge-Scaler Feature Mismatch:** Frequently encountered ValueError: The feature names should match those that were passed during fit.
- Reason: The StandardScaler was fitted on 10 features, but predict.py was passing an incorrect number or order of features.
- **Solution**: Always reconstructed the full 10-column DataFrame in the exact training order and applied scaling on the entire row.
- **Challenge:** Difference in XGBoost Performance: XGBoost achieved the highest F1-score of 0.9783 during model evaluation. However, the final model training resulted in slightly different performance (F1-Score (0.9663), ROC-AUC (0.9968), Accuracy (0.9647)).
- **Solution :** The final model was retrained separately, introducing slight randomness. Understood the reason behind variation and that small variation is normal in tree-based models. I decided to keep the model as it is and chose the best model from model comparison.
- **Challenge:** Label encoding in training vs manual mapping in prediction often caused errors or wrong results.
- **Solution:** Saved the actual LabelEncoder objects (le_dict) and reused them in predict.py for consistency.
- **Challenge:** The dataset (from Kaggle) was synthetic and had few biases, it contributed to limitations in the result of the model.
- **Learning:** Understood that the results depend on the dataset quality and hence access to diverse, real-world datasets is essential.

These challenges gave me hands-on experience with train-inference consistency, model persistence, and debugging real ML pipeline issues and these are the skills that go beyond just achieving high accuracy.

LIMITATIONS

While developing and testing the model I detected the presence of certain limitations in my project Hypertension Risk Prediction System. Identifying these limitations, searching and analyzing their causes was an important aspect in the development of the project. This entire process tested my problem identification and analysis skills. It also prompted me to think critically and interrogate my model.
The limitations which I could identify during the entire development and testing process are mentioned below:

- **Feature imbalance:** Some features, such as blood pressure history, smoking status, dominate the model's decisions, while lifestyle factors like exercise level contribute very little which is unrealistic. This can lead to unrealistic outputs. The model ignores or underweights certain features which are of importance in decision making, that's a sign of poor feature importance distribution. Protective factors (such as non-smoker) may be dominating the prediction. The model is over-relying on BP_History and that's why for an individual with Normal BP history but high stress/salt intake it is showing Low risk.
- **Synthetic dataset:** The data used was synthetic and **does not** reflect real medical records. As a result, the predictions may **not** align with real world outcomes. Also, it seems like the training dataset is biased (not enough young individuals with high stress/salt intake represented).
- **Probability calibration:** The model sometimes produces extreme probabilities (e.g., 0% or unexpectedly high values) that do not match medical reasoning. This is a common issue when models are trained on limited or imbalanced datasets. In most ML models (logistic regression, random forest, XGBoost), probabilities rarely hit exactly 0 or 1 hence it suggests that model may be miscalibrated or rounding aggressively.
- Hypertension Risk Prediction System is developed purely as an academic project to demonstrate the application of machine learning techniques. The outputs generated by this system are for educational purposes only and should not be interpreted as medical advice, diagnosis, or a substitute for professional healthcare. It is for educational purposes only and not a medical tool.

CONCLUSION

This project successfully demonstrates the application of supervised machine learning to solve a real-life problem. By building a complete pipeline from data preprocessing to inference, I gained practical experience with scikit-learn, XGBoost, data encoding, scaling, model persistence, and CLI development.

As my first project in the field of AI-ML, this project encouraged me to search for a meaningful problem in my environment and think critically about its solution and its implementation. It helped me explore, learn and utilise the various tools, techniques and concepts of Machine Learning in problem solving. It helped me understand the importance of data quality and its role in working/decision-making of a model , data preprocessing techniques. I learned how to train, evaluate and compare multiple models, analyze feature importance, and develop an inference system.

Throughout the project, I faced several challenges such as train-inference mismatches, scaler errors, and ensuring consistency between training and prediction pipelines. Solving these issues significantly improved my problem-solving skills and taught me the critical importance of reproducibility in machine learning systems. It taught me to think critically while developing a project and interrogating and validating the model.

Overall, this project has strengthened my foundation in Machine Learning concepts and boosted my confidence in applying theoretical knowledge to practical applications.